SOFTWARE REQUIREMENTS SPECIFICATION

IEEE 830 Standard Format

Project: Smart Expense & Budget Management Application

Version: 1.2

Date: 02 October 2026

Status: Final

Group Number: G-15

Authors:
Rhea Sushanth – PES1UG24CS371
Sarthki Vachhani – PES1UG24CS422
Sai Manogna BV – PES1UG24CS399

This document (IEEE 830 structure, following the course SRS template) specifies the functional, non-functional and security requirements of the Smart Expense & Budget Management Application, and expands on the problem statement and objectives introduced in the Synopsis.

Revision History

Version	Date	Author	Change Summary
1.0	07-09-2026	Group G-15	Initial submission (Deliverable 2 – Requirements)
1.1	02-10-2026	Group G-15	Resubmission restructured to the course SRS template: front matter, renumbered sections, Quality Attributes section, explicit Security Objectives, redrawn UML use-case and ER diagrams plus a second detailed use-case diagram, numbered RTM, Other Requirements moved to Appendix F, measurable values added to NFR-001, NFR-002, SEC-003, SEC-004 and SEC-006, four data fields added to Appendix B
1.2	02-10-2026	Group G-15	Consistency review against the Architecture & Design Specification v1.1 and Test Plan v1.1. Requirements clarified or extended: FR-001, FR-007, FR-012 to FR-014, FR-017, FR-019 to FR-021, FR-024, NFR-001, NFR-003, NFR-005, SEC-006. Appendices A to C: Active Budget, CSV layout, budget period and overlap rules, category type rule. Section 6 exit criteria aligned with the Test Plan; account deletion declared out of scope; Gantt chart redrawn; figures numbered.

Approvals

Role	Name	Signature/Date
Prepared by (Auth, Security requirements)	Sarthki Vachhani	Sarthki, 02-10-2026
Prepared by (Budget, Reporting, NFRs)	Rhea Sushanth	Rhea, 02-10-2026
Prepared by (Transactions, Import, Sandbox requirements)	Sai Manogna BV	Manogna, 02-10-2026

Table of Contents

1. Introduction

2. Overall description

3. External interface requirements

4. System features (detailed)

5. Non-functional requirements (detailed)

6. Quality attributes & Acceptance tests

7. System models and diagrams

8. Requirements Traceability Matrix (RTM)

Appendix A: Glossary

Appendix B: Data layout

Appendix C: Business rules

Appendix D: Validation specification

Appendix E: Project plan

Appendix F: Other requirements

1. Introduction

1.1 Purpose

The purpose of this document is to specify, in detail, the functional and non-functional requirements of the Smart Expense & Budget Management Application. It defines what the system shall do so that the development team, reviewers and evaluators share one unambiguous reference for design, implementation, and testing. This SRS expands on the problem statement and objectives introduced in the Synopsis.

1.2 Scope

The system in scope for this project is a single-user personal finance web application. It allows a user to record expenses and income, organize them by category, define budgets, receive threshold alerts, import transactions from CSV files, and view analytics dashboards. An administrator role manages the master category list at project level.

Scope (In):

- Single-user personal finance tracking (not multi-user shared households/accounts).
- Manual and CSV-based transaction entry; sandbox/dummy bank-linking as a stretch feature only.
- Category-wise budgeting, threshold alerts, and analytics/reporting dashboards.

Scope (Out):

- Real bank account linking or live payment processing.
- Multi-currency conversion or investment/portfolio tracking.
- Multi-user shared budgets (e.g., family accounts) — noted as a possible future enhancement.
- Account deletion by the user — not offered in this release.

1.3 Audience

Readers of this SRS: the development team (Rhea Sushanth, Sarthki Vachhani, Sai Manogna BV), course instructors/evaluators, and future maintainers, who use this document for design, evaluation, and implementation reference.

1.4 Definitions

See Appendix A: Glossary for the full list of terms and abbreviations (Transaction, Category, Budget, Threshold, Recurring Rule, Alert, Active Budget, RBAC, CSV Import).

References:

- Deliverable 1 - Synopsis / Project Proposal, Group G-15, "Smart Expense & Budget Management Application".

- IEEE Std. 830-1998 - Recommended Practice for Software Requirements Specifications (structure reference).
- Ian Sommerville, "Software Engineering", 10th Edition, Pearson Education, 2016 - course textbook, Unit 1 (Requirements Engineering).
- PES University, Software Engineering UE23CS341A — Unit 1 (Requirements Engineering: Elicitation, Analysis, Specification, Validation, RTM) and Unit 2 (Quality Attributes, Verification & Validation) course material.
- RFC 4180, "Common Format and MIME Type for Comma-Separated Values (CSV) Files," IETF, October 2005 — referenced for CSV import validation (FR-009, FR-010).
- OWASP Top 10 (2021), OWASP Foundation — referenced for security requirement categories (Section 5.1).
- G-15 Software Architecture & Design Specification v1.1 and G-15 Test Plan v1.1 (companion documents).

2. Overall Description

2.1 Product Perspective

The Smart Expense & Budget Management Application is a new, standalone web application. It is not a component of an existing system, though it is designed so that a sandboxed/mock bank-linking service and CSV import may be added as optional data sources without changing the core domain model. The application follows a client-server architecture: a Django backend exposing server-rendered views (with HTMX for partial updates) and a PostgreSQL/SQLite data store.

2.2 Product Functions

- Account registration, login and profile management.
- Recording and categorizing expense and income transactions.
- Importing transactions in bulk from a CSV file.
- Creating, editing and deleting category-wise budgets for a defined period and tracking adherence.
- Raising threshold and over-budget alerts/notifications.
- Generating spending reports and an analytics dashboard (charts).
- Configuring recurring transactions that auto-generate entries on schedule.
- Administrator management of the category master list and basic user oversight.

2.3 User Classes and Characteristics

User Class	Privileges	Characteristics
Customer (User)	Register, log in, record transactions, set budgets, view reports, receive alerts, manage own profile	Non-technical; expects a fast, mobile-friendly, low-friction flow for everyday logging
Administrator	Add/edit/disable/re-enable expense & income categories; basic user list oversight	Project-level trained user; infrequent but higher-privilege interaction

2.4 Operating Environment

- Server: Python 3 with the Django web framework, deployed behind a WSGI/ASGI server.
- Database: PostgreSQL in staging/production; SQLite for local development.
- Client: Any evergreen desktop or mobile web browser (Chrome, Edge, Firefox, Safari) rendering Django templates enhanced with HTMX and Chart.js.
- Version control and CI: Git/GitHub with GitHub Actions running automated tests on each push.

2.5 Constraints

- Web application only for this iteration.
- No real bank account linking or live payment/settlement processing is permitted; only sandbox/dummy data sources.
- Single currency, single-user accounts — no multi-currency conversion and no shared/family budgets.

2.6 Assumptions and Dependencies

- Users have access to a modern web browser and stable internet connectivity; the application is not designed for offline use.
- The chosen tech stack (Python/Django) works well, so no additional framework-training time is budgeted into the sprint plan.
- The sandbox/dummy bank-linking stretch feature, if attempted, depends on the availability of a free/test-tier third-party API; if no suitable sandbox API is available, this feature will be dropped without affecting core scope.
- Continuous Integration (GitHub Actions) is assumed to be available and unrestricted for the team's repository throughout the project timeline.

3. External Interface Requirements

3.1 User Interfaces

- Responsive web UI built with Django templates, HTMX (for partial page updates without full reloads) and Chart.js (for report visualizations).
- Core screens: Login/Register, Dashboard (summary + charts), Add/Edit Transaction, Budgets, Reports, CSV Import, Admin Category Management.
- Forms shall show inline validation errors (e.g., invalid amount, missing category) before submission where feasible.

3.2 Hardware Interfaces

Not applicable. The application is a browser-based web system with no dedicated hardware component; it runs entirely on standard server and client computing hardware already covered under the Operating Environment (Section 2.4).

3.3 Software Interfaces

- Backend framework: Django (Python), exposing server-rendered views and HTMX partial endpoints.
- Database interface: Django ORM against PostgreSQL (production) or SQLite (development).
- Charting library: Chart.js, fed with JSON data prepared by Django views.
- CSV import interface: server-side parser (Python csv module / pandas) validating uploaded files against the expected schema (column layout in Appendix B.8).
- Sandbox bank-linking interface via a mock third-party REST API, isolated behind an adapter so it can be swapped or disabled.

3.4 Communications Interfaces

- HTTPS for all client-server communication.
- Session-based authentication for authenticated requests.
- HTMX/AJAX partial responses are returned as HTML fragments; JSON is used only for chart data payloads.

4. System Features (detailed)

Requirement-writing rule: each requirement is uniquely identified and satisfies the properties of a good requirement emphasised in this course — Concise (describes a single property), Clear and Unambiguous, Verifiable/Measurable, Feasible, Traceable and Necessary. For example, a vague statement such as "the system must be user-friendly" is unverifiable; it is refined here into concrete, testable requirements such as FR-003 (profile editing) and NFR-003 (usability, measured in number of screens to complete a task).

Priority follows the MoSCoW scale (Must have / Should have / Could have / Won't have) taught in the course, mapped to High = Must have, Medium = Should have and Low = Could have below; items that would fall under Won't-have are already excluded under Out of Scope (Section 1.2).

4.1 Feature: User Authentication & Profile Management (owner: Sarthki Vachhani)

ID	Requirement	Priority	Verification
FR-001	The system shall allow a user to register an account using a unique email address and a password of at least 8 characters containing at least one letter and one digit.	High	Functional test
FR-002	The system shall allow a registered user to log in using their email and password.	High	Functional test
FR-003	The system shall allow an authenticated user to view and edit their profile information.	Medium	Functional test
FR-004	The system shall allow a user to log out, invalidating the active session.	High	Functional test

4.2 Feature: Expense / Income Transaction Management (owner: Sai Manogna BV)

ID	Requirement	Priority	Verification
FR-005	The system shall allow an authenticated user to record an expense transaction with amount, category, date and an optional note.	High	Functional test
FR-006	The system shall allow an authenticated user to record an income transaction with amount, category, date and an optional note.	High	Functional test
FR-007	The system shall require every transaction to be assigned exactly one active category of the same type (Expense or Income) before it can be saved.	High	Negative + positive test
FR-008	The system shall allow a user to edit or delete a transaction they created.	Medium	Functional test
FR-009	The system shall allow a user to import transactions in bulk from a CSV file matching the documented column layout (Appendix B.8).	High	Integration test
FR-010	The system shall validate an imported CSV file and report any malformed rows without importing them, while importing all valid rows.	High	Negative + positive test
FR-011	The system shall detect and skip duplicate transactions during CSV import, based on matching date, amount and note, whether the match is with an existing transaction or with an earlier row of the same file.	Medium	Integration test

4.3 Feature: Budget Setting, Alerts & Reporting/Analytics (owner: Rhea Sushanth)

ID	Requirement	Priority	Verification
FR-012	The system shall allow a user to create, edit and delete a budget for a category over a defined period (weekly or monthly).	High	Functional test
FR-013	The system shall calculate and display cumulative expense spend (Expense transactions only) against each active budget, updated within 2 seconds of a transaction being saved.	High	Unit + system test
FR-014	The system shall reject creating or editing a budget so that it overlaps another budget of the same user for the same category (overlap rule in Appendix B.3).	High	Negative test
FR-015	The system shall notify a user when spend reaches a configurable threshold percentage (default 80%) of a budget.	High	Functional test
FR-016	The system shall notify a user when spend exceeds 100% of a budget.	High	Functional test
FR-017	The system shall generate a spending report grouped by category for a selected time period, and a budget adherence report showing the limit, spent-to-date and remaining amount for each active budget.	High	System test
FR-018	The system shall display an analytics dashboard with chart-based visualizations (pie, bar, line) of spending trends.	Medium	System test
FR-019	The system shall allow a user to configure, edit and deactivate a recurring transaction rule (amount, category, frequency) that automatically creates transactions on schedule.	Medium	Integration test

4.4 Feature: Administrator / Category Management (owner: shared)

ID	Requirement	Priority	Verification
FR-020	The administrator shall be able to add, edit, disable and re-enable expense/income categories.	Medium	Functional test

ID	Requirement	Priority	Verification
FR-021	The system shall prevent a disabled category from being assigned to new transactions, budgets or recurring rules.	Medium	Negative test
FR-022	The administrator shall be able to view a basic list of registered users for project-level oversight.	Low	Functional test

4.5 Feature: Sandbox Bank-Linking (stretch feature, owner: shared)

ID	Requirement	Priority	Verification
FR-023	The system shall allow a user to optionally sync transactions from a sandbox/mock bank-linking service, subject to third-party sandbox API availability.	Low	Integration test (mocked)

4.6 Feature: Scheduled Transaction Generation (owner: shared)

This feature separates the automatic, Scheduler-triggered side of recurring transactions (UC-11) from the User-configured setup (UC-08, FR-019).

ID	Requirement	Priority	Verification
FR-024	The system shall automatically generate a transaction from an active recurring rule (FR-019) on its scheduled run date, without user action. If the rule's category has been disabled, the run shall skip that rule, flag it on the user's recurring page and continue with the other rules.	High	Integration test

5. Non-Functional Requirements (detailed)

NFRs below are measurable and tied to the Test Plan. Non-functional requirements are further classified into product requirements (how the delivered product must behave), organisational requirements (a consequence of team/process policy) and external requirements (arising from factors outside the system and its development process).

ID	Category	Measurable Requirement	Verification	Classification
NFR-001	Performance	95% of dashboard/report requests shall receive a response within 2 seconds under a test load of 50 concurrent users with 10,000 stored transactions, with an error rate below 1%.	Performance test	Product requirement
NFR-002	Availability	The application shall be available at least 99% during the operating window of 06:00–24:00 IST, excluding planned maintenance.	Operational review	Product requirement
NFR-003	Usability	A new user shall be able to record a basic expense in no more than 3 primary screens, without help.	Usability walkthrough	Product requirement
NFR-004	Reliability	A CSV re-import of the same file shall not create duplicate transactions.	Negative/ integration test	Product requirement

ID	Category	Measurable Requirement	Verification	Classification
NFR-005	Maintainability	Business logic (budgeting, alerts, recurrence) shall be separated into identifiable Django apps/modules with automated unit tests reaching at least 70% line coverage on the budgets, recurring and transactions apps.	Code review + coverage report	Organisational requirement — team coding/module standard
NFR-006	Testability	Core services (budget calculation, alert evaluation) shall expose testable interfaces that do not require an external bank/payment sandbox to run.	Unit/integration test	Organisational requirement — team testing standard
NFR-007	External Policy	The application shall be developed and hosted in a manner consistent with campus network and academic data-handling policy.	Compliance/ operational review	External requirement

5.1 Security

5.1.1 Security Objectives

The system is validated against three security objectives, consistent with the companion Test Plan (Section 5.1):

Objective	Meaning for this application
Confidentiality	A user can see only their own financial data, and stored passwords are unreadable.
Integrity	Only the owner can change a transaction, budget or rule, and uploaded input cannot alter the system.
Accountability and access control	Admin functions are restricted, sessions expire, and security events are logged.

5.1.2 Security Requirements

ID	Security Requirement	Validation Approach
SEC-001	Passwords shall not be stored in plaintext; they shall be hashed using a strong, salted algorithm.	Code/configuration review
SEC-002	Role-based access control shall prevent customers from accessing administrator functions.	Authorization tests
SEC-003	The system shall validate and sanitize all user-supplied fields, including uploaded CSV content, before processing; CSV uploads shall be limited to 2 MB.	Negative/security tests
SEC-004	Authenticated endpoints shall reject expired or invalid session/credentials; a session shall expire after 30 minutes of inactivity.	Security test
SEC-005	Sensitive financial identifiers (e.g., sandbox bank tokens) shall not be stored unnecessarily by the application.	Design review
SEC-006	Security-relevant events such as failed login attempts, unauthorized access attempts and administrator category changes shall be logged, and repeated failed login attempts shall be throttled: 5 failed attempts within 10 minutes shall lock the account for 10 minutes.	System test

6. Quality Attributes & Acceptance Tests

- Exit criteria for acceptance: all High-priority functional requirements implemented and verified, no critical NFR failures, and the full RTM (seeded in Section 8 and completed in the Test Plan, Section 13.3) shows every requirement mapped to at least one test case, with all High-priority requirements passing and the overall results meeting the release-exit criteria in the companion Test Plan (Section 14).
- Acceptance test suites: Authentication & Profile, Transactions & CSV Import, Budgets & Alerts, Reports & Dashboard, Recurring Transactions, Admin & Category Management, Security, and Performance/Usability — matching the feature groupings in Section 4 and the Test Plan's Section 3 (Features to be Tested).
- Quality attributes prioritized for this project: Security and Reliability (financial data, no duplicate transactions) and Maintainability/Testability (small team, 12-week schedule) are treated as High priority throughout; Performance and Usability are Medium/High. Each attribute is made measurable by the NFR and SEC requirements in Section 5, and their test cases are traced in Section 8.

7. System Models and Diagrams

7.1 UML Use-Case Diagram

Figure 1 shows the four actors (User (Customer), Administrator, Payment/Bank Sandbox (external system) and Scheduler (system clock/cron)) and the use cases each can invoke. It uses standard UML use-case notation: actors as stick figures, use cases as ovals inside the system boundary, and `<<include>>` and `<<extend>>` dependency arrows. The `<<triggers>>` arrow is a project-specific stereotype showing the Scheduler starting UC-11.

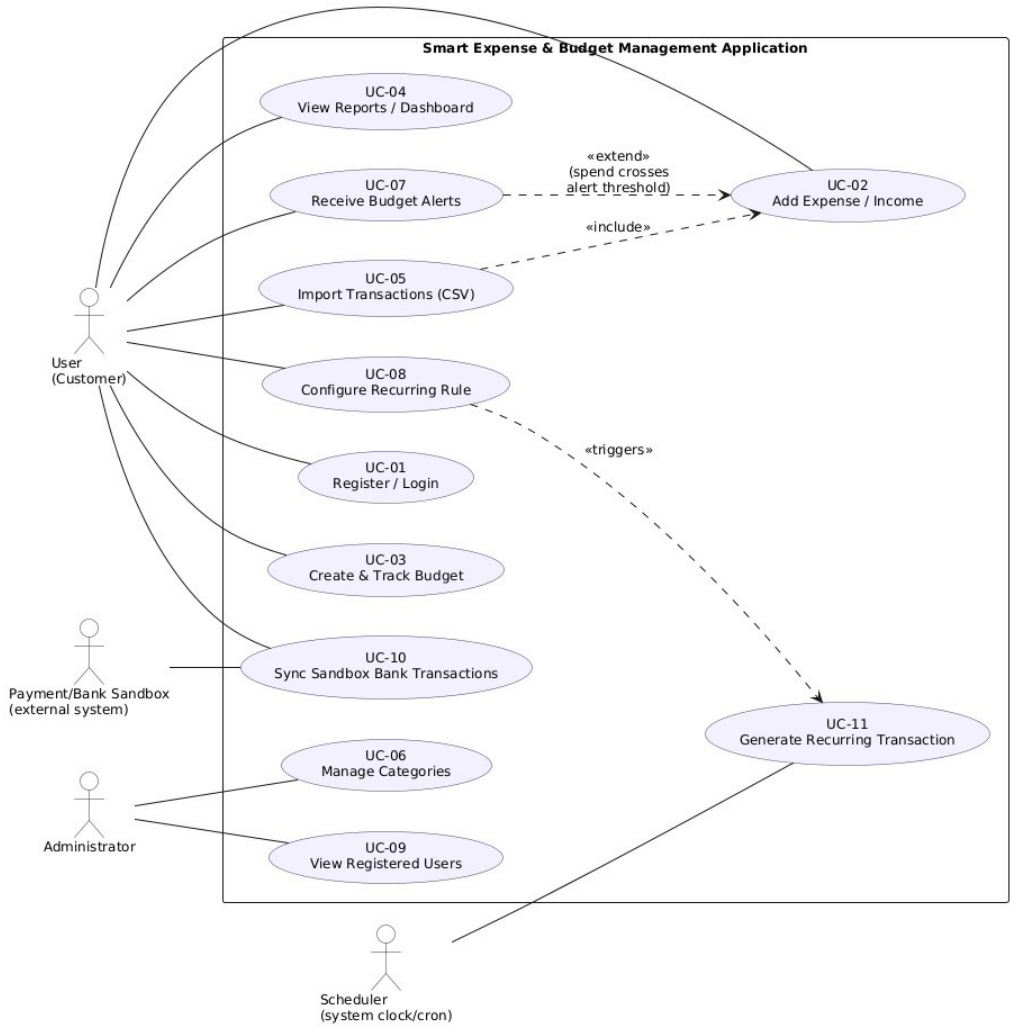


Figure 1: System-level UML use-case diagram, showing all 11 use cases and 4 actors

Use Case ID	**Use Case**	**Primary Actor**	**Precondition**	**Outcome**
UC-01	Register / Login	User	Application available	Authenticated session
UC-02	Add Expense / Income	User	Logged in	Transaction recorded
UC-03	Create & Track Budget	User	Logged in; category exists	Budget created / spend tracked
UC-04	View Reports / Dashboard	User	Logged in; transactions exist	Report/dashboard displayed
UC-05	Import Transactions (CSV)	User	Logged in; valid CSV file	Valid rows imported
UC-06	Manage Categories	Administrator	Admin authenticated	Category list updated
UC-07	Receive Budget Alerts	User	Active budget exists	Alert/notification shown
UC-08	Configure Recurring Rule	User	Logged in	Recurring rule scheduled
UC-09	View Registered Users	Administrator	Admin authenticated	User report displayed

Use Case ID	Use Case	Primary Actor	Precondition	Outcome
UC-10	Sync Sandbox Bank Transactions	User	Logged in; sandbox bank-linking enabled	Transactions imported from sandbox
UC-11	Generate Recurring Transaction	Scheduler	Active recurring rule exists; scheduled run date reached	Transaction created automatically

Figure 2 zooms into the transaction and budget management use cases (UC-02, UC-03, UC-05, UC-07, UC-08, UC-10, UC-11). It shows the same `<<include>>`, `<<extend>>` and `<<triggers>>` relationships as Figure 1 and the same actors except the Administrator; the administrator and login use cases are left out.

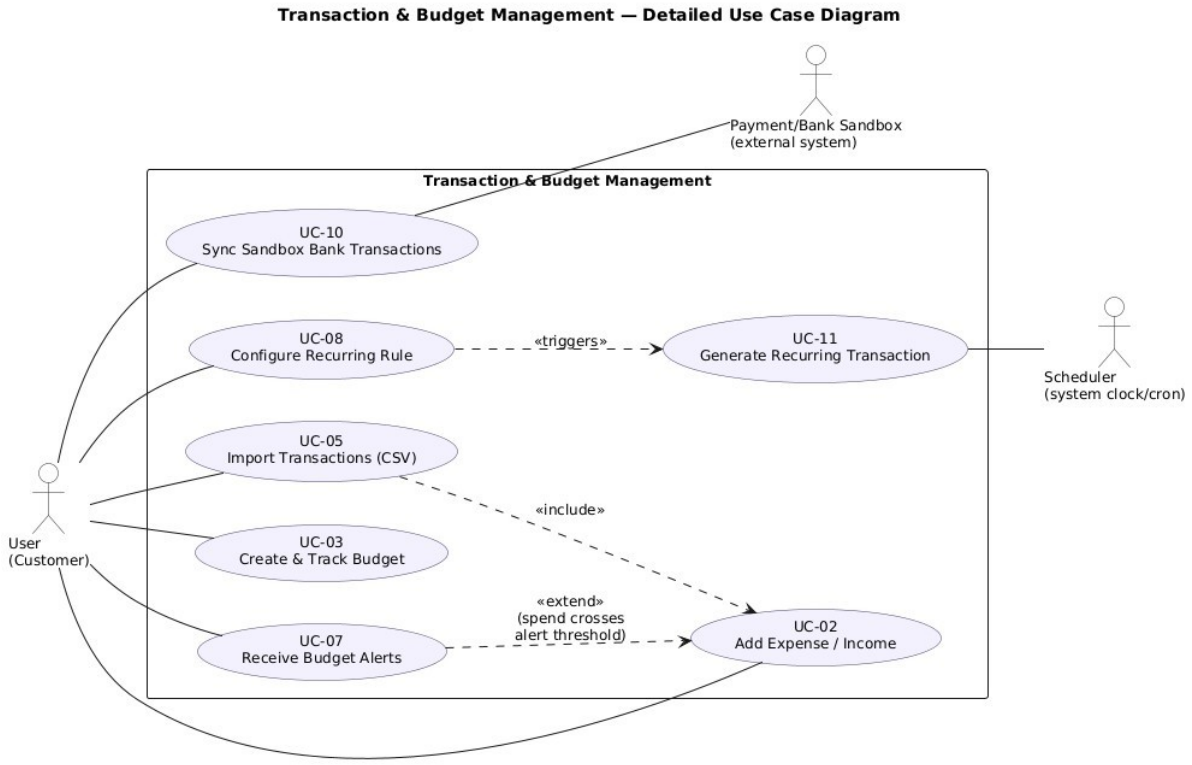


Figure 2: Detailed UML use-case diagram for transaction and budget management

7.2 Domain / Entity-Relationship Model

The core domain entities — User, Category, Transaction, Budget, Recurring Rule and Alert — and their relationships.

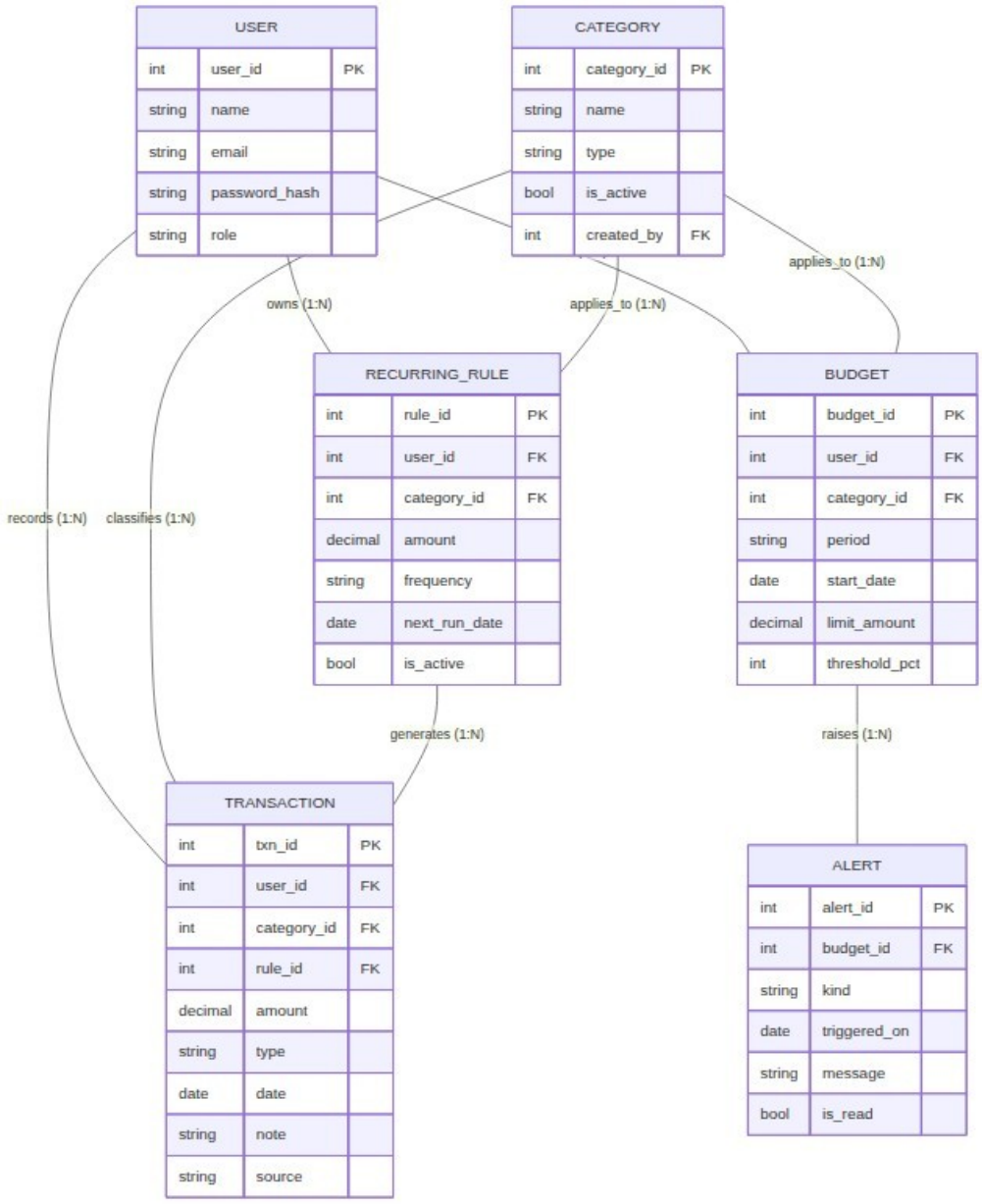


Figure 3: Entity-relationship diagram of the Smart Expense & Budget Management Application's six core domain entities

8. Requirements Traceability Matrix (RTM)

Following the RTM approach, requirements are traced across the SDLC in both directions — forward tracing (requirement → architectural section → design section → implementation unit → test ID) and backward tracing (test → requirement) — with every phase progressively filling in the matrix. It is seeded here at the requirements stage and is extended by the companion Architecture & Design Specification and Test Plan.

Req ID	Related Use Case	Architectural Section	Design Section	Implementation Unit	Test ID	Status (N/P/A)
FR-001	UC-01	Auth Service	accounts app — registration view	`accounts/views.py`	TC-AUTH-01	N
FR-002	UC-01	Auth Service	accounts app — login view	`accounts/views.py`	TC-AUTH-02	N
FR-005	UC-02	Transaction Service	transactions app — create view	`transactions/models.py`	TC-TXN-01	N
FR-009	UC-05	Import Service	transactions app — CSV importer	`transactions/importer.py`	TC-TXN-05	N
FR-012	UC-03	Budget Service	budgets app — budget model	`budgets/models.py`	TC-BUD-01	N
FR-015	UC-07	Alert Service	budgets app — alert evaluator	`budgets/alerts.py`	TC-BUD-04	N
FR-017	UC-04	Reporting Service	reports app — category and budget reports	`reports/views.py`	TC-RPT-01	N
FR-019	UC-08	Scheduler Service	recurring app — rule runner	`recurring/tasks.py`	TC-REC-01	N
FR-020	UC-06	Admin Service	categories app — admin view	`categories/admin.py`	TC-ADM-01	N
FR-023	UC-10	Bank Sandbox Adapter	transactions app — sandbox adapter	`transactions/sandbox_adapter.py`	TC-TXN-06	N
FR-024	UC-11	Scheduler Service	RuleRunner class	`recurring/tasks.py`	TC-REC-02	N
SEC-001	UC-01	Auth Service	accounts app — password hashing	`accounts/models.py`	TC-SEC-01	N
SEC-002	UC-06	Security layer	accounts app — permissions	`accounts/permissions.py`	TC-SEC-02	N
NFR-001	UC-04	Reporting Service	reports app — caching/query plan	`reports/queries.py`	TC-PERF-01	N

Status codes: N = not yet executed, P = passed, A = accepted at the sprint demo. Status is N for every row because no test case has been executed at the requirements stage. This seed set is extended to all 37 requirement-to-test mappings in the companion Test Plan, Section 13.3.

Appendix A: Glossary

Term	Definition
Transaction	A single recorded expense or income entry with amount, category, date and optional note.
Category	A classification label (e.g., Food, Rent, Salary) applied to a transaction; can be expense- or income-type.
Budget	A user-defined spending limit for a category over a period (weekly/monthly).
Active Budget	A budget whose period (Appendix B.3) has not yet ended, i.e. it covers today or a later date.
Threshold	A configurable percentage of a budget (default 80%) at which an alert is raised.
Recurring Rule	A schedule that automatically generates a transaction at a fixed frequency (e.g., monthly).
Alert	A notification raised when spend reaches or exceeds a budget threshold.

Term	Definition
RBAC	Role-Based Access Control - restricting functionality based on assigned role (Customer/Administrator).
CSV Import	Bulk upload of transactions from a comma-separated-values file matching a documented schema.

Appendix B: Data Layout

B.1 User Fields

Field	Type	Notes
user_id	Integer (PK)	Auto-generated
name	String	Required
email	String (unique)	Required; used for login
password_hash	String	Never stored/displayed in plaintext
role	Enum (Customer/Administrator)	Default: Customer

B.2 Transaction Fields

Field	Type	Notes
txn_id	Integer (PK)	Auto-generated
user_id	Integer (FK)	Owner of the transaction
category_id	Integer (FK)	Must reference an active category of the same type as the transaction
rule_id	Integer (FK, optional)	Set only when the transaction was generated by a recurring rule (FR-024)
amount	Decimal	> 0; required
type	Enum (Expense/Income)	Required
date	Date	Required; defaults to today
note	String	Optional, max 250 characters; stored as an empty string when omitted, so that duplicate matching (FR-011) never compares null values
source	Enum (Manual/CSV/Recurring/Sandbox)	Set automatically by the entry path

B.3 Budget Fields

Field	Type	Notes
budget_id	Integer (PK)	Auto-generated
user_id	Integer (FK)	Owner
category_id	Integer (FK)	Must reference an active Expense category
period	Enum (Weekly/Monthly)	Required
start_date	Date	Required; first day of the budget period, used for the overlap check (FR-014)
limit_amount	Decimal	> 0; required
threshold_pct	Integer	Default 80; range 1-100

Period rule: a Weekly budget covers start_date through start_date + 6 days (7 days). A Monthly budget covers start_date through the day before the same day-of-month in the following month (for example 15 Sep to 14 Oct; if that day does not exist in the following month, the budget runs through that month's last day). Two budgets overlap when they are for the same category and their date ranges share at least one day, whatever their period type (so a weekly budget cannot sit inside a monthly one for the same category).

B.4 Category Fields

Field	Type	Notes
category_id	Integer (PK)	Auto-generated
name	String	Required
type	Enum (Expense/Income)	Required
is_active	Boolean	Default: true; disabled and re-enabled by Administrator (FR-020)
created_by	Integer (FK → User)	Administrator who created the category

B.5 Recurring Rule Fields

Field	Type	Notes
rule_id	Integer (PK)	Auto-generated
user_id	Integer (FK)	Owner
category_id	Integer (FK)	Must reference an active category; generated transactions take their type from it
amount	Decimal	> 0; required
frequency	Enum (Weekly/Monthly)	Required
next_run_date	Date	Auto-updated after each generated transaction
is_active	Boolean	Default: true; set to false when the user deactivates the rule, which stops further generation

B.6 Alert Fields

Field	Type	Notes
alert_id	Integer (PK)	Auto-generated
budget_id	Integer (FK)	Related budget
kind	Enum (Threshold/Limit)	Threshold = spend reached threshold_pct (FR-015); Limit = spend exceeded 100% (FR-016)
triggered_on	Date	Auto-set when threshold/limit is crossed
message	String	Auto-generated notification text
is_read	Boolean	Default: false

B.7 Report Requirements

- Category-wise spend report for a selected date range, with total and percentage of overall spend.

- Budget adherence report showing limit, spent-to-date and remaining amount per active budget.
- Trend chart (line) of daily/weekly spend over the selected period.
- Category breakdown chart (pie/bar) for the selected period.

B.8 CSV Import Layout

Uploaded files shall be UTF-8 encoded, at most 2 MB, with a header row exactly as below (RFC 4180 conventions). The source field of every imported transaction is set to CSV.

Column	Type	Rules
date	Date	Required; ISO format YYYY-MM-DD
amount	Decimal	Required; greater than 0; up to 2 decimal places
type	Text	Required; Expense or Income (case-insensitive)
category	Text	Required; must match the name of an active category of the same type
note	Text	Optional; maximum 250 characters

A row is a duplicate when its date, amount and note match an existing transaction of the same user, or an earlier row of the same file (a missing note is treated as an empty string).

Appendix C: Business Rules

- Only an authenticated user may create, edit or delete their own transactions, budgets and recurring rules.
- A transaction must belong to exactly one active category, and the category type must match the transaction type.
- A budget may be set only on an Expense category and counts Expense transactions only.
- A recurring rule has no type of its own: each transaction it generates takes its type from the rule's category.
- A user cannot have two overlapping budgets for the same category (overlap rule in Appendix B.3).
- Only an administrator can disable or re-enable a category.
- A disabled category cannot be assigned to a new transaction, budget or recurring rule.
- A recurring rule generates a new transaction only on its scheduled run date, and stops generating once deactivated.
- CSV import shall not create duplicate transactions for rows already present in the system or repeated within the same file.

Appendix D: Validation Specification — Initial Planning

At this requirements stage, the validation specification is planning-focused: it explains how functional, security and non-functional requirements will eventually be validated, including planned security testing. This is not the final test report (see the companion Test Plan for that).

Validation vs. Verification: As distinguished in the course: Validation determines whether the requirements, if implemented, will solve the right problem and satisfy the intended user's needs. Verification determines whether the requirements have been specified correctly. Reviews are used for both.

Planned Validation Techniques:

- Requirement reviews: peer walkthroughs within the team, and instructor/evaluator review at each deliverable checkpoint.
- Prototyping: low-fidelity mockups of the transaction-entry and dashboard screens, which involve the most user interaction, to confirm the team and stakeholders share the same interpretation before implementation.
- Model validation: checking that the use-case model (Section 7.1) and the ER/domain model (Section 7.2) are internally consistent and jointly cover every functional requirement in Section 4.
- Acceptance-criteria checks: matching each sprint's user-story acceptance criteria (Appendix E) against the functional requirement(s) it is meant to satisfy.

Validation Plan by Requirement Category:

Requirement Category	Planned Validation	Example
Functional	Black-box functional tests	Valid/invalid expense entry, budget creation
Security	Authentication/authorization testing + planned penetration testing (e.g., OWASP-style checks for injection, broken access control, and auth bypass)	Customer attempts to access admin category endpoint
Performance	Load/response-time test	Concurrent dashboard/report requests
Usability	Task-based walkthrough	New user records a basic expense
Reliability	Failure/retry scenarios	Re-importing the same CSV file
Unit	Automated unit tests + coverage	Budget spend and alert-threshold calculation

Appendix E: Project Plan

E.1 Lifecycle Justification

Agile (Scrum-style sprints) fits this project because requirements around budgeting and reporting features are likely to evolve as user feedback comes in during demos. Short, fixed-length sprints let the team deliver working increments early starting with login in Sprint 1, gather feedback, and adjust scope — e.g. deciding how deep to go on sandbox bank-linking without blocking the whole project on upfront design of every feature.

E.2 Reuse vs. Build Classification

Component	Classification	Notes
Django web framework	Reuse	Off-the-shelf framework, not built by the team
Django's built-in authentication (password hashing, sessions)	Reuse	Used as-is for FR-001–004, SEC-001
Chart.js	Reuse	Off-the-shelf charting library for FR-017 and FR-018
HTMX	Reuse	Off-the-shelf library for partial page updates
Django Admin (base)	Reuse (developer tool)	Built-in admin site kept for developers only, at `/django-admin/` and limited to superusers; FR-020 to FR-022 are delivered by team-built, role-protected views in the categories app
Accounts app (registration/profile logic)	Build	Custom Django app, team-written
Transactions app (expense/income + CSV import)	Build	Custom Django app, team-written
Budgets app (budget tracking + alerts)	Build	Custom Django app, team-written
Recurring app (recurring transaction scheduler)	Build	Custom Django app, team-written
Reports app (analytics/dashboard queries)	Build	Custom Django app, team-written
Categories app (category master list, role-protected admin views)	Build	Custom Django app, team-written

E.3 Work Breakdown Structure

WP	Tasks	Owner	Est. Effort (person-days)	Output
WP1	Requirements baseline + use cases	All	4	SRS
WP2	Architecture, ER model, UML	All	4	Architecture/ Design
WP3	Auth & profile module	Sarthki Vachhani (SV)	6	Working feature
WP4	Expense/income transaction module	Sai Manogna BV (SM)	7	Working feature
WP5	Budget creation & tracking module	Rhea Sushanth (RS)	6	Working feature
WP6	Reports & analytics dashboard	Rhea Sushanth (RS)	6	Working feature
WP7	Alerts, recurring transactions (incl. scheduler-triggered generation, FR-024), admin panel, sandbox bank integration (FR-023, stretch)	SV / SM / RS	7	Working feature
WP8	Testing, CI/CD, security hardening	All*	6	Test/validation evidence
WP9	Final validation & report	All	4	Final report

**Testing can be split among the team.*

E.4 Sprint Schedule / Gantt Chart

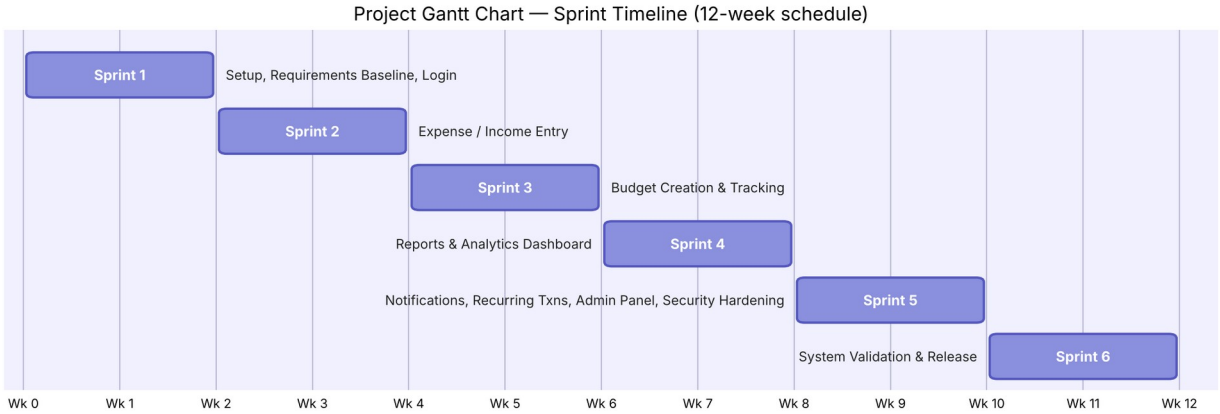


Figure 4: Project Gantt chart showing the 6-sprint, 12-week schedule (sprints run back to back: weeks 0-2, 2-4, 4-6, 6-8, 8-10, 10-12)

E.5 Tool Stack

Activity	Recommended / Example Tool	Evidence to Submit
Backend	Python (Django)	Repository code, running app
Database	PostgreSQL / SQLite (dev)	Schema/migration files
Frontend	Django templates + HTMX + Chart.js	Screens, chart demo
Version control	Git + GitHub	Repository URL, commit history
Agile planning	GitHub Projects	Backlog, user stories, sprint board
CI	GitHub Actions	Pipeline run log
Testing	Pytest / Django test framework	Test results
API/UI testing	Postman / Django test client	Collection/results
Design	PlantUML / Mermaid.js	UML / ER diagrams

Appendix F: Other Requirements

- Database: relational schema (PostgreSQL/SQLite) with referential integrity between User, Category, Transaction, Budget, Recurring Rule and Alert.
- Data retention: transaction and budget history shall be retained for the lifetime of the user account. Account deletion is not offered in this release (Section 1.2).
- Legal/compliance: no real financial credentials are collected; the application explicitly avoids scope requiring PCI-DSS or banking-regulation compliance.
- Localization: single language (English) and single currency for this iteration; the data model should not preclude future localization.
- Reuse: authentication, RBAC and CSV parsing are implemented as reusable Django apps/modules where practical.